RouterAcc: FPGA Acceleration for VLSI Detailed Router via Hierarchical Storage Mapping

***Abstract*—Detailed routing constitutes a critical phase in the very large-scale integration (VLSI) physical design, widely regarded as the most time-consuming and computationally intensive step in the back-end design process. Due to its iterative characteristics and strong data dependencies, conventional parallel computing acceleration techniques often prove difficult to apply and yield limited effectiveness. To address these challenges, we propose RouterAcc, an FPGA-based software–hardware co-designed acceleration framework tailored for VLSI detailed routing. RouterAcc incorporates access analysis and a termination condition to accelerate convergence. We further employ hierarchical storage mapping and a flexible dimension-partitioning architecture to alleviate memory bottlenecks and enhance data locality. Moreover, RouterAcc leverages a hierarchical comparison pipeline with fully parallelized computing units and a data preprocessing strategy to further improve computational efficiency. Experimental results on the ISPD'18 benchmarks demonstrate that RouterAcc achieves consistent speedups of 2.1×–2.3× over TritonRoute with less than 1% quality degradation. With further co-optimization, RouterAcc attains speedups of 2.7×–11.8× while maintaining routing quality comparable to TritonRoute and surpassing Dr.CU 2.0 as well as the state-of-the-art (SOTA) FPGA-based approaches.**

I. Introduction

In the computationally intensive process of integrated circuit (IC) physical design, routing constitutes a critical stage, establishing functional electrical connections among components while adhering to strict manufacturability constraints. Given the immense scale of modern design, routing is conventionally divided into two phases: global routing, which provides coarse-grained path guidance, and detailed routing, which performs fine-grained assignments of wires and vias under stringent design rules. While global routing has been extensively studied with optimization strategies to constrain the search space and reduce computational overhead [1]–[5], detailed routing remains a major bottleneck [6]. It often consumes the majority of runtime due to its complex path exploration, parasitic resistance-capacitance (RC) delay modeling, crosstalk analysis, and frequent rule-checking requirements.

Accelerating detailed routing is particularly challenging due to iterative dependencies and memory-intensive algorithms, which make traditional parallel computing approaches ineffective. Detailed routing algorithms, *e.g.*, TritonRoute [7] and Dr. CU 2.0 [8], face substantial performance limitations stemming from memory access patterns. These limitations are most pronounced in the maze routing kernel, commonly based on A* or Dijkstra search algorithm. One of the main bottlenecks in these algorithms lies in the management of the open, closed lists, and priority queue during the search process. In particular, operations on the open list dominate execution time in single-threaded implementations [9], and even parallelization fails to overcome the memory-bound nature of the algorithm.

To accelerate routing algorithms, prior work has explored heterogeneous platforms such as GPUs. In VLSI design, however, acceleration has focused primarily on global routing using GPUs [2] [3] [10] [11], as detailed routing is more challenging due to large problem sizes and complex design rules. Recently, Field-Programmable Gate Arrays (FPGAs) have emerged as a promising platform for detailed routing acceleration [12], [13]. Unlike CPUs and GPUs, FPGAs provide abundant on-chip memory and fine-grained parallelism, which are critical for reducing memory latency and enabling efficient large-scale data interactions. Leveraging on-chip cache, FPGAs can mitigate the memory bottleneck of external memory access, reducing both latency and power consumption. Several studies have investigated FPGA acceleration for detailed routing. For example, [13] introduced predictive prefetching and crossbar-based parallel cost calculation, achieving a 3.1× speedup with minimal quality degradation. However, this approach did not fully exploit FPGA programmability and customizability.

In this paper, we propose RouterAcc, a software–hardware co-design framework for FPGA-accelerated detailed routing. RouterAcc integrates pin accessibility analysis and an intelligent termination mechanism to reduce redundant rip-up-and-reroute iterations. To alleviate memory access bottlenecks, we introduce a memory-efficient architecture with hierarchical storage mapping and reconfigurable dimension partitioning. The main contributions are summarized as follows:

- We propose RouterAcc, an FPGA-based software-hardware co-design acceleration framework tailored for the detailed routing phase of VLSI physical design.
- We integrate pin access analysis and present a termination condition to prevent redundant rip-up-and-reroute iterations, and adapt the routing algorithm to FPGA hardware.
- We present hierarchical storage mapping and flexible dimension partition architecture to overcome the detailed routing acceleration bottleneck. Moreover, we introduce a hierarchical comparison pipeline and fully parallelized computing units to improve computational efficiency.
- We demonstrate that RouterAcc achieves consistent 2.1×–2.3× acceleration of TritonRoute with negligible quality loss. In addition, a co-optimization strategy further improves performance to 2.7×–11.8×, delivering quality comparable to TritonRoute while outperforming Dr.CU 2.0 and prior FPGA accelerators.

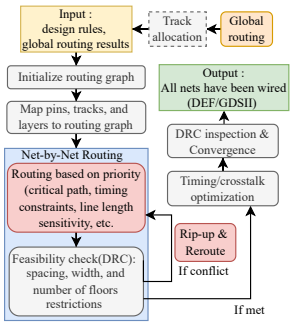


Fig. 1: Classic industrial detailed routing Flow.

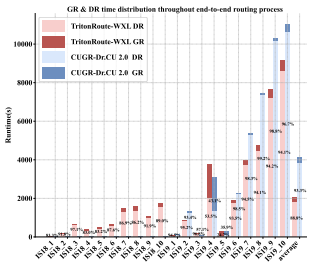


Fig. 2: Runtime proportion of GR and DR.

II. Background and Motivation

A. Detailed Routing Flow

Detailed routing refines the results of global routing at a finer level of granularity. The classic industrial detailed routing flow is shown in Fig. 1. The routing resources contain a stack of metal layers, where wire segments (tracks) run horizontally or vertically on each layer. Vias are introduced to connect wires across adjacent metal layers. The routing grid is typically abstracted as a three-dimensional graph $G(V, E)$, where V denotes the set of vertices corresponding to track locations separated by fixed distances. E denotes the set of edges representing wire segments or vias that connect the vertices on the grid graph. A detailed router parses industrial layout design files, *e.g.*, LEF, DEF, and guide files, generates exact rectilinear interconnects, and determines wire paths and via placements. During this process, it enforces complex design rules and supports multi-dimensional analyses, such as parasitic RC delay modeling and crosstalk evaluation. To accomplish these tasks, modern detailed routing algorithms often employ maze-based search with heuristic cost functions, followed by iterative rip-up-and-reroute procedures to resolve violations.

B. Bottleneck

1) Urgency of Accelerating Detailed Layout: In advanced full-process global–detailed routing (GR–DR) flows, *e.g.*, TritonRoute-WXL [6] and the CUGR [5]-and-Dr.CU 2.0 [8] flow, detailed routing accounts for approximately 90% of the total runtime, while global routing contributes only about 10%, as shown in Fig. 2. This statistic is based on the acceleration of detailed routing using CPU multi-threading. Under standard single-threaded operations, the proportion of detailed routing is higher. Due to its vast path search space, iterative operations, and frequent design rule checks on high-resolution databases, detailed routing dominates runtime. While global routing acceleration has been extensively studied [1], [2], [4], [11], detailed routing remains difficult to accelerate because of its strong data dependencies.

2) Memory Access Bottleneck: Classical detailed routers rely primarily on two data structures: the open list and the closed list [14]. The closed list, often implemented with a linked hash table, prevents redundant state expansion, while

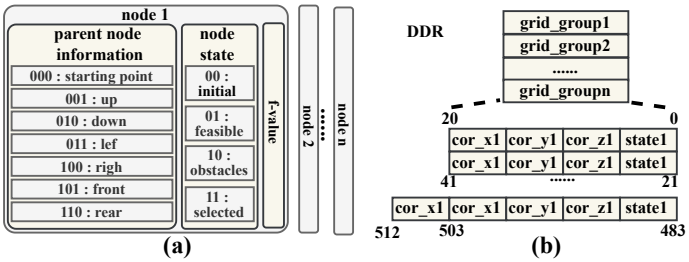


Fig. 3: FPGA data structure. (a) Information stored in a RAM block. (b) Data organization in DDR.

the open list, typically managed by a binary heap, prioritizes states using heuristic cost values. As reported in [9], operations on the open list account for approximately 95% of execution time in single-threaded implementations, and even with parallelization across eight threads, this proportion only decreases to 88%. The persistent dominance of memory operations highlights a fundamental architectural limitation, *i.e.*, computational optimizations cannot overcome bottlenecks arising from memory-bound execution and inefficient resource utilization. This challenge is exacerbated in complex environments with high obstacle density, where frequent cache misses further degrade performance predictability and compromise real-time responsiveness.

C. Motivation

Recent studies have explored FPGA acceleration for detailed routing [12] [13], employing predictive prefetching, parallel computing units, and crossbar structures to achieve 3.1× speedup with minimal quality degradation. However, such straightforward parallel designs have not fully exploited the programmability and customizability of FPGAs. By leveraging these features together with real-time pipeline operations, it becomes possible to tailor and optimize back-end detailed routing algorithms and flows, thereby unlocking greater acceleration potential and achieving higher performance.

III. Algorithm Optimization

A. Data Structure on FPGA

In CPU-based maze routing, data structures are inefficient for FPGA accelerators. To address this, we redesign the data layout. As shown in Fig. 3(a), the on-chip RAM stores compact node information, enabling fast access for parallel expansion. Fig. 3(b) shows the DDR organization, where grid groups record the coordinates and state of nodes, allowing efficient large-scale data access and supporting fine-grained parallelism.

B. Detailed Routing Flow Optimization

The overall framework of our detailed routing algorithm is presented in Algorithm 1 and Fig. 4(a). The routing flow begins with database initialization on the CPU, followed by the construction of the grid structure (GCells) [16] and pin accessibility analysis [6]. Pre-routed nets are initialized, and pin access points are collected to facilitate subsequent routing operations (Line 1). During the rip-up-and-reroute stage

Algorithm 1 Iterative Optimization of Dtailed Routing

Input: benchmark files
Output: full connected nets
1: InitDb(), InitPreRouteNet(), PinAccessAnalysis()
2: **for each** SearchRepair stage **do**
3: **Select** all $drcNets$ from $nets$
4: **Partition** $drcNets$ into batches $\{batch_1 \ldots\ldots batch_n\}$
5: **for** $net \in batch_i$ **in Parallel do** ▷ per search parallel
6: $ucPins \leftarrow$ GetPins(net)
7: **while** $ucPins \neq \emptyset$ **do**
8: $tpin \leftarrow \underset{pin \in ucPins}{\arg\max}\ pin.cost$
9: $cPins \leftarrow cPins \cup tPin,\ ucPins \leftarrow ucPins \setminus tPin$
10: $G \leftarrow$ PrepareGraph($cPins, ucPins$)
11: FPGA-OptimizedRoutingKernel(G)
12: **end while**
13: UpdateDRC(), UpdateNets()
14: **end for**
15: **end for**
16: UpdateDb()

Algorithm 2 FPGA-Optimized Routing Kernel Algorithm

Input: graph batch G_k
Output: connected nodes set
1: **Init** Hierarchical RAM of existing nodes
2: **while** $node \in$ feasible nodes **do**
3: Update $node$ information in RAM
4: $p_nodes \leftarrow$ **Expand** $node$ in six directions
5: Upload $node \rightarrow$ astar_top
6: **for** $n \in p_nodes$ **in Parallel do** ▷ per direction parallel
7: **if** $v \in G_k.end_points$ **then**
8: Upload $p_node \rightarrow$ astar_top **and Exit**
9: **end if**
10: Calculate f_cost_value and g_cost_value
11: Compare intra_RAM $nodes$ cost_min
12: Update n information intra_RAM
13: **end for**
14: Compare inter_RAM cost_min
15: **end while**

(Lines 2-15), the CPU sends batches of routing instructions to the FPGA, which executes parallelized search operations and returns results to the CPU via the top-level module. This iterative process progressively reduces design rule violations (DRCs) and converges toward an optimized detailed routing solution. To further improve efficiency, we integrate the state-of-the-art pin access analysis [15], which jointly accounts for track resources, pin accessibility, and design rules, thereby enhancing congestion estimation and layer allocation. In addition, when the DRC count is already low or its reduction stagnates, repeated iterations tend to re-examine only a small number of violations while incurring up to 50% runtime overhead. To avoid unnecessary computation, we introduce an early-termination condition to prevent redundant iterations.

For each main iteration loop (Lines 2-14), nets with unresolved violations are divided into batches and routed in parallel. Within the inner loop (Lines 7-12), the algorithm repeatedly selects the pin with the highest routing cost from the set of unconnected pins. The FPGA-optimised routing kernel (Line 11) computes paths between pins while updating the routing graph G (Line 10), which maintains connectivity and cost metrics. The routing graph is also designed to capture routing stats, including the grid graph, source pin, connected components, and tracking metrics (pathData, perTime, and stepCnt). Once all SearchRepairs stages are completed or convergence is reached, the final results are committed, and the routing process is terminated.

C. FPGA-Optimized Routing Kernel Algorithm Flow

The conventional maze routing algorithm is adapted to the characteristics of the FPGA, as illustrated in Algorithm 2. The grid map is first initialized on the CPU, and then the data is transmitted to the DDR on the FPGA for storage. The routing kernel begins by initializing the hierarchical RAM with node data. From each node, potential routing paths are expanded in six directions, *i.e.*, up, down, left, right, forward, and rear (Lines 2-5). These six expanded nodes are processed simultaneously within a computation module that supports six degrees of parallelism (Lines 6-13). If a target endpoint is reached during the expansion (Lines 7-9), the corresponding node is immediately passed to astar_top, and the search for that net is terminated.

For each neighbor node, the algorithm evaluates the heuristic cost (Line 10) and performs a two-stage cost comparison. 1) Intra-RAM comparison (Line 11): row-level comparators within each storage unit first identify local minima, and a depth-first tree arbiter then selects the optimal candidate. 2) Inter-RAM comparison (Line 14): the global minimum across RAMs is chosen. If the updated cost is smaller than the previous result from the binary-tree comparator, it is directly selected; otherwise, the prior node is retained and the new value is inserted for a partial tree update. Finally, the updated node states are written back to RAM (Line 12), maintaining the address-to-coordinate mapping required to sustain hardware-based data flow.

IV. Hardware architecture

A. Overview of RouterAcc Framework

An overview of the CPU–FPGA architecture of RouterAcc is shown in Fig. 4. The CPU dispatches batches of routing instructions to the FPGA, which executes parallelized search operations and returns results via the top-level module. Before execution, the preprocessed layout is partitioned into GCell units, and the associated data structures are reformatted into an FPGA-friendly representation before being transferred. On the FPGA, the computationally intensive iterative rip-up-and-reroute operations are performed, and the final routing results are sent back to the CPU.

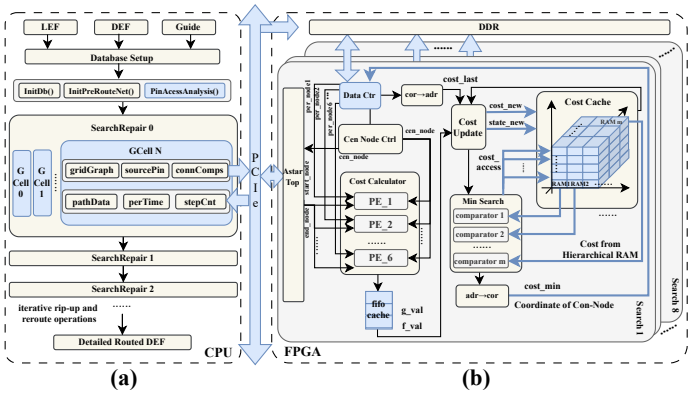


Fig. 4: RouterAcc framework. (a) CPU-based detailed routing design flow. (b) FPGA-embedded acceleration framework.

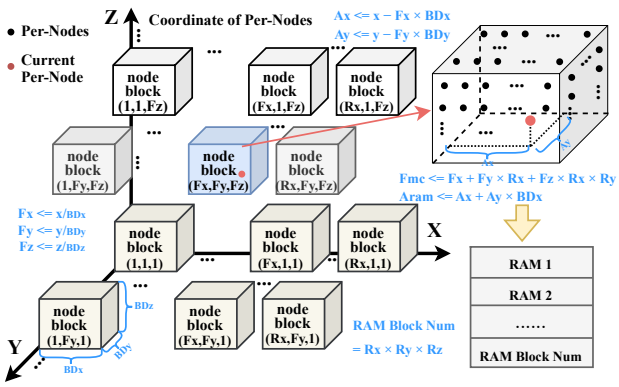


Fig. 5: Hierarchical RAM blocks mapping.

The Asta_top module receives routing instructions, specifies the start_node and end_node, and monitors the overall process. The search begins from the start_node (initialized as the central node, cen_node), which is inserted into the open list. The system then fetches prefetched node data from data_ctr, examines the peripheral nodes (per_nodes) of the cen_node, and adds reachable ones to the open list. The cen_node is subsequently moved to the close list to avoid reprocessing.

Path cost computation is handled by the cost_calculator module, where clusters of processing elements (PEs) independently access graph data from data_ctr and evaluate, in parallel, the costs from per_nodes to the end_node. The cost_update module then invokes the hierarchical comparison pipeline (min_search) to update f-values, validate costs, and prevent anomalies. Storage units are selectively updated, and the node with the smallest f-value is selected as the next cen_node.

During exploration, cen_nodes are iteratively expanded, with their per_nodes added to the open list, while f-values of all open nodes are recalculated and updated. The exploration ends when all nodes are correctly connected. The above modules collectively form a search core, and the architecture instantiates as many cores as CPU threads, enabling parallel exploration and significantly improving routing efficiency.

B. Hierarchical Storage Mapping

1) Hierarchical RAM Blocks Instantiation: Traditional FPGA-based path searching suffers from severe inefficiencies in dynamic data management, particularly in maintaining open and closed lists. Dynamic sorting of these data structures requires frequent reordering after each search iteration [14], which consumes substantial programmable logic resources and introduces processing delays that are misaligned with hardware execution, thereby degrading real-time performance. Moreover, the dynamic memory management of variable-length data structures leads to fragmented Block RAM access, leading to reduced resource utilization.

To address these challenges, we propose a hierarchical RAM mapping strategy that organizes storage by dimensional priority, as illustrated in Fig. 5. For two-dimensional data, each RAM module independently stores nodes by row, while in the three-dimensional case, each RAM module is assigned to a fixed plane. An address mapping module converts high-dimensional coordinates, *e.g.*, 3D coordinates (x, y, z) in this work, into two-dimensional storage addresses within each RAM module. This approach improves storage access efficiency and enhances overall throughput.

We denote $BD_{\mathrm{x}}, BD_{\mathrm{y}}, BD_{\mathrm{z}}$ as the number of data elements in the three dimensions of a RAM block, respectively. The three-dimensional enable signals F_{x}, F_{y}, F_{z} specify the coordinates of the RAM that stores the current node within the RAM matrix. The RAM matrix dimension D_{ram} is $R_{\mathrm{x}} \times R_{\mathrm{y}} \times R_{\mathrm{z}}$, and the number of RAM blocks is calculated and divided based on this. The RAM enable signal is F_{mc}. Within this matrix, the address of a node in one RAM along the x direction is denoted as A_{x}, along the y direction as A_{y}, and the physical RAM access address as A_{ram}. The mapping expressions from node coordinates to RAM addresses are illustrated in Fig. 5.

To further improve efficiency, we design a flexible dimension division mechanism that adaptively partitions RAM resources according to routing characteristics. Initially, partitioning along the y–z plane resulted in long computation cycles due to frequent traversal across this plane. To this end, we introduce two improvements: 1) changing the RAM partitioning plane from y–z to x–y to reduce access overhead, and 2) applying finer meshing within the x–y plane to decrease the number of nodes per RAM block, thereby compressing the maximum period required for minimum value search within each RAM.

2) State-driven Update Control: As shown in Fig. 3(a), nodes are distributed across RAM blocks according to their coordinates. Each node stores a parent, state, and f-value, all encoded in binary. The encoded data is written to the assigned RAM address in a single operation After computing the f-values for neighboring nodes, the corresponding storage entries are updated under logical control. The `cost_cache` retrieves the node's state and historical f-value. If the state is initial (00), the new value is written directly. If the state indicates feasibility (01), a hardware comparator updates the entry only when the new f-value improves path quality. This state-driven conditional write strategy reduces redundant memory access and overwriting, while ensuring preservation of path optimality.

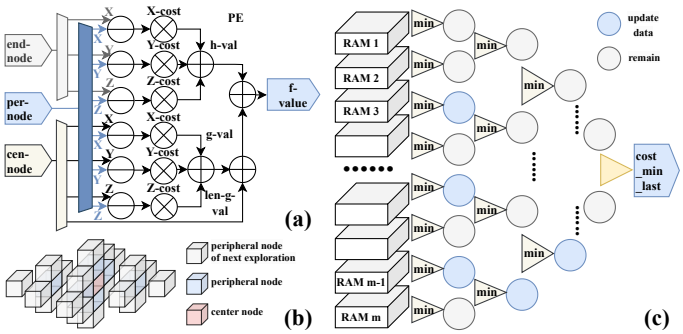


Fig. 6: (a) The PE of cost_calculator module. (b) Memory manager prefetches from DDR in advance to reduce memory access latency. (c)Workflow of the `cost_compare` module.

C. Hardware-Optimized Execution on FPGAs

Algorithm 2 and Fig. 4(b) illustrate the detailed routing algorithm adapted for FPGA-based hardware-optimized execution.

1) Low-Latency Cost Calculation: Each routing direction is computed in parallel by separate processing elements (PEs), as shown in Fig. 6(a), thereby accelerating wavefront expansion. Within the `Data_ctr` module, peripheral nodes in six directions are prefetched together with the nodes of the next iteration, as shown in Fig. 6(b). This pipelined prefetching ensures that the required data is already available during computation, effectively hiding memory access latency.

2) Hierarchical Comparison Pipeline Architecture: In distributed storage architectures, selection of the least-cost node is achieved through a hierarchical self-updating comparison mechanism. Each storage unit integrates a hardware minimum-value tracker, which triggers parallel comparison whenever a new f-value is detected. Register groups are employed to dynamically update the minimum cost and corresponding 3D coordinates, while a valid status flag is sent to the arbitration controller. As illustrated in Fig. 6(c), a double-buffer register is employed to decouple computing and storage units, ensuring continuous pipeline operation during data updates. This distributed arbitration mechanism transforms exhaustive search into multi-level pipelining, mitigating resource contention in centralized comparison and enabling real-time path planning.

V. Experimental Results

A. Experimental Setup

Hardware Implementation. The acceleration framework is implemented on a Linux system with an 8-core Intel(R) Xeon(R) Gold 5218R CPU@2.10GHz with 64 GB memory, and a Xilinx XCVU9P FPGA with 1.18M LUTs, 2.36M FlipFlops, 2.16K Block RAMs, and 960 Ultra RAMs.

Benchmarks. We evaluate our framework using the ISPD'18 detailed routing benchmarks [17]. To ensure fairness, TritonRoute [7] and Dr.CU 2.0 [8] are re-executed on our server, and their detailed routing kernel runtimes are recorded. We also report the performance of the state-of-the-art FPGA-based accelerator in [13]. Since its implementation is not open source, we could not re-evaluate it on our platform. Therefore, we compare acceleration ratios rather than raw runtimes to avoid bias caused by differences in CPU platforms.

As shown in Fig. 7(a), the runtime of the batched maze routing kernel with 8 CPU threads approaches the peak performance of a multicore CPU. Accordingly, we test TritonRoute and Dr.CU 2.0 with 8 threads on our CPU server and configure 8 parallel pipelines on the FPGA for acceleration. It is worth noting that [13] adopted 12 PEs for acceleration, while also using the 8-thread CPU runtime as the baseline.

B. Acceleration Efficiency

Table I summarizes the performance comparison among TritonRoute (TR) [7], Dr.CU 2.0 (CU) [8], the SOTA FPGA-based accelerator ([13]-CU), our FPGA-accelerated TritonRoute(RA-TR), and our software-hardware co-optimized framework (CoRA-TR), which first optimizes the detailed routing algorithm and then performs hardware acceleration.

ISPD'18 quality scores indicate that TR generally achieves superior routing quality compared to CU and its accelerated variants, consistent with using multiple rip-up-and-reroute iterations to refine solutions. Our FPGA-accelerated version, RA-TR, maintains quality close to TR, with variations typically within 1%, attributed to minor precision loss during hardware execution. CoRA-TR, which further integrates pin access analysis, achieves the best overall results, maintaining quality comparable to TR across different benchmarks. After accelerating the routing process, our results outperform [13]-CU in routing quality on most test cases.

Under 8-thread CPU parallelism, TritonRoute achieves strong performance. We accordingly configured our FPGA accelerator with eight parallel pipelines for fair comparison. This setting yields a stable acceleration of 2.1-2.3× across all test cases, with negligible quality degradation. Given that the CPU frequency (2.10 GHz) is approximately 14× higher than that of our FPGA (150 MHz), the obtained acceleration clearly demonstrates the efficiency of our design.

The iteration count of TritonRoute is approximately 3× that of Dr.CU 2.0 [8]. To further improve acceleration, we incorporate pin access analysis from TritonRoute-WXL [6] to optimize the selection of source and sink nodes. Through such software–hardware co-optimization, CoRA-TR surpasses Dr.CU 2.0 in speed, achieving additional improvements in acceleration ratio. Although our exploration iterations remain nearly twice those of Dr.CU, the vast majority of cases still outperform [13], achieving 2.7×–11.8× speedups. Future refinements in source and sink node selection are expected to reduce iterations and enhance acceleration efficiency.

C. Scalability Exploration of Multiple PEs

As shown in Fig. 7(a), the average runtime with 8 threads nearly reaches the peak performance on a multi-core CPU. On the FPGA, performance increases almost linearly as the number of parallel PEs grows from 1 to 16. And the acceleration ratio can continue to improve with additional hardware resources, indicating strong scalability of our framework on larger FPGA platforms.

TABLE I: Performance comparison of TritonRoute [7], Dr.CU 2.0 [8], and our FPGA-accelerated framework on the batched maze routing kernel using the ISPD 2018 [17] contest benchmarks.

Benchmark	ISPD'18 Quality Score†					Detailed Routing Kernel (8 thread*)						
						Runtime(s)				Speed-up		
	TR	CU	[13]-CU	RA-TR	CoRA-TR	TR	CU	RA-TR	CoRA-TR	[13]-CU	RA-TR	CoRA-TR
ISPD18_testl	304934	291305	290099	**303483**	**307257**	4.41	1.59	**2.03**	**1.6**	2.0×	**2.2**×	**2.7**×
ISPD18_test2	4831983	4700581	4685023	**4810886**	**4819019**	58.86	18.57	**25.93**	**18.68**	2.2×	**2.3**×	**3.1**×
ISPD18_test3	5283329	5295331	5291929	**5281145**	**5332504**	110.07	25.60	**52.4**	**52.93**	2.1×	**2.1**×	**2.1**×
ISPD18_test4	15035816	15761114	15808947	**15082968**	**14957868**	244.48	120.63	**211.68**	**20.56**	2.4×	**2.3**×	**11.8**×
ISPD18_test5	16077963	16370540	16406741	**16103496**	**15971928**	109.42	86.68	**40.39**	**25.20**	2.7×	**2.3**×	**4.3**×
ISPD18_test6	21352959	21636718	21666362	**21367147**	**21083855**	165.39	125.90	**77.33**	**45.36**	2.7×	**2.1**×	**3.6**×
ISPD18_test7	38148393	38408621	38523171	**38206329**	**37867997**	246.12	234.84	**111.98**	**88.77**	2.8×	**2.2**×	**2.8**×
ISPD18_test8	38307564	38571533	38717711	**38346227**	**38207086**	256.73	212.6485	**125.29**	**94.63**	2.8×	**2.1**×	**2.7**×
ISPD18_test9	32925038	33114610	33342024	**33067470**	**32805346**	264.21	158.15	**128.19**	**83.10**	2.9×	**2.1**×	**3.1**×
ISPD18_test10	40403574	40998097	42820500	**40528793**	**41254471**	380.69	345.35	**180.455**	**189.812**	2.7×	**2.1**×	**2.0**×

* Except for [13]-CU, which adopts 12 degrees of parallelism, all other works adopt 8 degrees of parallelism.

† The ISPD'18 Quality Score evaluates routing solution quality, with a lower score indicating better performance, adjusted for penalties like non-determinism.

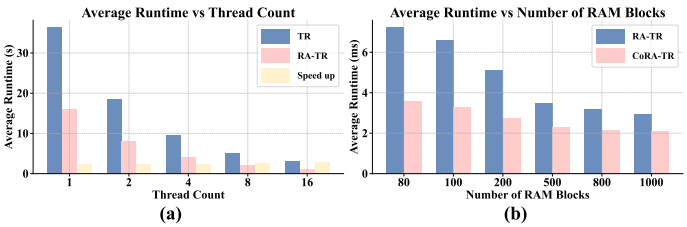


Fig. 7: Impact of RAM block partitioning and thread count on acceleration performance.

TABLE II: Hardware resource utilization.

Resource	Utilization	Available	Percent (%)
LUT	291736	1182240	24.67
FF	226588	2364480	9.58
BRAM	1200	2160	55.55
URAM	242	960	25.20

D. Design Space Exploration of the RAM Number

Due to limited resources, we utilized 100 RAM matrices, each of size 10×10, for the entire experiment. To investigate the impact of RAM quantity on acceleration, we tested the first 300 search processes of `ispd8_test1`, collected FPGA simulation data, measured runtimes under different RAM configurations, and normalized the results by reporting runtime per 10,000 explored nodes.

As shown in Fig. 7(b), the performance of our FPGA acceleration framework improves as the number of RAM blocks increases. When sufficient on-chip resources are available, employing finer-grained RAM partitioning to increase the number of blocks enhances storage comparison and update throughput, thereby achieving better acceleration.

These results suggest that acceleration performance can be further improved on FPGA boards with more abundant resources. However, unlimited subdivision is impractical due to the inherent limits of on-chip resources. Once the partitioning granularity exceeds a certain threshold, the marginal performance gain diminishes. Therefore, selecting an appropriate RAM partitioning granularity according to the specific routing scale is essential to achieve strong acceleration while avoiding resource waste from excessive subdivision.

E. Consumption of Hardware Resource

Table II summarizes hardware utilization. RAM resources are maximized to construct larger matrix units, leading to relatively high memory utilization. Memory dominates detailed routing and becomes the primary bottleneck as benchmarks scale. By leveraging high on-chip bandwidth, FPGAs effectively reduce latency in data-intensive operations, making them a promising platform for routing acceleration.

VI. Conclusions

In this paper, we presented RouterAcc, an FPGA-based acceleration framework for detailed routing that addresses the memory access bottleneck through hierarchical storage mapping, state-driven updates, and a hierarchical comparison pipeline. On ISPD'18 benchmarks, RouterAcc achieves consistent 2.1×–2.3× speedup of TritonRoute with negligible quality loss, and up to 11.8× when co-optimized. Compared with Dr.CU 2.0 and prior FPGA accelerators, RouterAcc delivers both higher performance and competitive solution quality. These results demonstrate the feasibility and promise of FPGA-based acceleration for breaking the long-standing runtime bottleneck in detailed routing.

REFERENCES

[1] Wen-Hao Liu, Wei-Chun Kao, Yih-Lang Li, and Kai-Yuan Chao, "Multi-threaded collision-aware global routing with bounded-length maze routing," in *Proceedings of the ACM/IEEE Design Automation Conference (DAC)*, 2010.

[2] Wen-Hao Liu, Wei-Chun Kao, Yih-Lang Li, and Kai-Yuan Chao, "NCTU-GR 2.0: Multithreaded collision-aware global routing with bounded-length maze routing," in *Proceedings of the ACM/IEEE Design Automation Conference (DAC)*, 2010.

[3] Y Han and K Chakraborty, and S Roy, "A global router on GPU architecture," in *Proceedings of the IEEE International Conference on Computer Design (ICCD)*, 2013.

[4] Jiayuan He, Martin Burtscher, Rajit Manohar, and Keshav Pingali, "SPRoute: A scalable parallel negotiation-based global router," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2019.

[5] JinWei Liu, Chak-Wa Pui, Fangzhou Wang, and Evangeline F. Y. Young, "CUGR: Detailed-Routability-Driven 3D Global Routing with Probabilistic Resource Model," in *Proceedings of the ACM/IEEE Design Automation Conference (DAC)*, 2020.

[6] Andrew B. Kahng, Lutong Wang, and Bangqi Xu, "TritonRoute-WXL: The open-source router with integrated DRC engine," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD). 2021, 1076-1089.

[7] Andrew B. Kahng, Lutong Wan,g and Bangqi Xu, "TritonRoute: The open-source detailed router," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD). 2021, 40(3): 547-559.

[8] Haocheng Li, Gengjie Chen, Bentian Jiang, Jingsong Chen, and Evangeline F. Y. Young, "Dr. CU 2.0: A Scalable Detailed Routing Framework with Correct-by-Construction Design Rule Satisfaction," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2019.

[9] Yuzhi Zhou, Xi Jin, and Tianqi Wang, "FPGA implementation of A* algorithm for real-time path planning," International Journal of Reconfigurable Computing, 2020, 1-11.

[10] Han, Y., Ancajas, D. M., Chakraborty, K., and Roy, S, "Exploring high-throughput computing paradigm for global routing," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2013, 22(1), 155-167.

[11] Shiju Lin, Jinwei Liu, Evangeline F. Y. Young, and Martin D. F. Wong, "GAMER: GPU-accelerated maze routing," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD). 2023, 42(2): 583-593.

[12] Korolija, Dario, and Mirjana Stojilović, "FPGA-assisted deterministic routing for FPGAs," in *Proceedings of the IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2019.

[13] Xun Jiang, Jiarui Wang, Yibo Lin, and Zhongfeng Wang, "FPGA-accelerated maze routing kernel for VLSI designs," in *Proceedings of the IEEE Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2022.

[14] Adham Osama, Ahmed Mostafa, Eslam Mamdouh, Mohamed Gamal, Usama Imam, Mohamed Taha, Ahmed Khalil, Islam Ahmed, and Hassan Mostafa, "Fast RTL implementation of A* path planning algorithm," in *Proceedings of the International Conference on Microelectronics (ICM)*, 2021.

[15] Andrew B. Kahng, Lutong Wang, and Bangqi Xu, "The Tao of PAO: Anatomy of a pin access oracle for detailed routing," in *Proceedings of the ACM/IEEE Design Automation Conference (DAC)*, 2020.

[16] Dolgov, Sergei, et al, "2019 cad contest: Lef/def based global routing," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2019.

[17] Stefanus Mantik, Gracieli Posser, Wing-Kai Chow, Yixiao Ding, and Wen-Hao Liu, "ISPD 2018 Initial Detailed Routing Contest and Benchmarks," in *Proceedings of the International Symposium on Physical Design (ISPD)*, 2018.